

INFO-H-502 – 3D Graphics Project Report

Space Game

Louis David Duman

May 2026

Introduction

The game implemented in this project is a 3D space shooter developed in OpenGL. The player pilots a spaceship in space and must avoid asteroids. The game combines interactive camera control, textured 3D models, collision detection, particle effects, and procedural asteroid generation to create an immersive space environment.

The source code is available on GitHub: <https://github.com/L6uis9/projet-info-h502>
All the references and sources used for this project are being mentioned in the `README.md`

The project is structured as follows:

- `main.cpp`: main game orchestration.
- `camera.cpp`: camera orientation and computation.
- `model.cpp`: OBJ/MTL parser with GPU upload.
- `asteroid.cpp`: asteroid related computation and generation.
- `particle.cpp`: implementation of the particle system.
- `postprocess.cpp`: off-screen FBO and motion blur.
- `skybox.cpp`: cubemap loading and skybox rendering.
- `star.cpp`: sun billboard.
- `shader.cpp`: GLSL shader compilation and uniform setters.
- `textureLoader.cpp`: image loading via `stb_image`.

The shaders are the followed:

- `model.vert/frag`: Phong lighting, triplanar mapping, and reflection/refraction.
- `motionblur.vert/frag`: radial motion blur.
- `particle.vert/frag`: instanced billboard quads for particles.
- `skybox.vert/frag`: skybox rendering.
- `star.vert/frag`: sun billboard.

1 Part 1 – Basic Features

1.1 Lighting

All lighting is computed per fragment in `shaders/model.frag` using the Phong model. The scene can contain up to `MAX_STARS` stars, implemented as light sources with *no attenuation*, to contribute to the final colour. In the final code, we have $N = 1$ star, but it can be extended to more stars. The total light is:

$$\mathbf{L}_o = \mathbf{L}_{\text{ambient}} + \sum_{i=1}^N \left(\mathbf{L}_{\text{diffuse},i} + \mathbf{L}_{\text{specular},i} \right)$$

Ambient

The ambient term approximates the ambient light and is independent of any light direction:

$$\mathbf{L}_{\text{ambient}} = \mathbf{k}_a \odot \mathbf{c}_{\text{base}}$$

where $\mathbf{k}_a = (0.005, 0.005, 0.008)$ simulates the darkness of space, and $\mathbf{c}_{\text{base}}$ is the surface colour. $\odot$ denotes component-wise multiplication.

Diffuse

The diffuse term depends on the angle between the surface normal $\hat{\mathbf{N}}$ and the vector $\hat{\mathbf{L}}_i$ pointing from the fragment toward the star i (The hat notation indicates a normalized vector, i.e. a vector of length 1):

$$\mathbf{L}_{\text{diffuse},i} = \mathbf{c}_{\text{light},i} \odot \mathbf{c}_{\text{base}} \cdot \max(0, \hat{\mathbf{N}} \cdot \hat{\mathbf{L}}_i)$$

The max clamps surfaces facing away from the light.

Specular

The reflected ray term is:

$$\hat{\mathbf{R}}_i = 2 (\hat{\mathbf{N}} \cdot \hat{\mathbf{L}}_i) \hat{\mathbf{N}} - \hat{\mathbf{L}}_i$$

The highlight intensity then depends on how closely the view direction $\hat{\mathbf{V}}$ aligns with $\hat{\mathbf{R}}_i$:

$$\mathbf{L}_{\text{specular},i} = \mathbf{c}_{\text{light},i} \cdot k_s \cdot \max(0, \hat{\mathbf{R}}_i \cdot \hat{\mathbf{V}})^\gamma$$

with specular coefficient $k_s = 0.4$ and shininess exponent $\gamma = 32$. The GLSL built-in `reflect()` implements $\hat{\mathbf{R}}_i$.

Light colour

Each star has a colour $\mathbf{c}_{\text{star},i}$. Rather than using it directly, the shader blends it with white to avoid having a tinted light:

$$\mathbf{c}_{\text{light},i} = (1 - \alpha) \mathbf{1} + \alpha \mathbf{c}_{\text{star},i}, \quad \alpha = 0.4$$

Normal transformation

Surface normals must be transformed by the *inverse-transpose* of the model matrix to remain perpendicular to the surface under non-uniform scaling:

$$\hat{\mathbf{N}} = (M^{-\top})_{3 \times 3} \hat{\mathbf{N}}_{\text{local}}$$

1.2 Textures

Textures are loaded from JPEG/PNG files using the `stb_image` library (`src/textureLoader.cpp`). The shader uses the uniform `hasDiffuse` to choose the base colour: if a diffuse texture (`map_Kd`) exists, it samples it, otherwise it uses the material colour `matKd`. See Section 3.1 for more details.

We use *triplanar mapping* to avoid texture distortion on procedurally generated asteroids. Three samples of the same texture are taken on three planes (YZ, XZ, XY). The surface colour of the asteroid is then:

$$\mathbf{c}_{\text{base}} = w_x \cdot T(\text{yz}) + w_y \cdot T(\text{xz}) + w_z \cdot T(\text{xy})$$

where $T(uv)$ is the texture colour at coordinates uv , and the weights $\mathbf{w} = (w_x, w_y, w_z)$ are defined as:

$$\mathbf{w} = \frac{|\hat{\mathbf{N}}_{\text{local}}|^4}{\|\|\hat{\mathbf{N}}_{\text{local}}\|^4\|_1}$$

The exponent 4 sharpens the blend.

1.3 Loading Multiple Models

The project loads several models at startup. File models use the custom OBJ/MTL parser in `src/model.cpp`, the implementation details are described in Section 3.1. Procedural asteroid meshes are described separately in Section 4.3.

The scene loads three distinct model types:

- **Spaceship:** `assets/models/spaceship.obj`, multi-mesh with per-material diffuse textures.
- **Earth:** `assets/models/Earth 2K.obj`, two meshes (*Earth* surface and *Clouds*).
- **Asteroids:** procedurally generated meshes.

1.4 Cubemap

The space background is rendered as a *skybox*: a large cube whose faces are textured with six images representing the six directions of the environment. The six faces are loaded as a `GL_TEXTURE_CUBE_MAP` in `src/skybox.cpp`.

The vertex shader removes the translation component of the view matrix (keeping only rotation) so the skybox does not translate as we move.

To ensure the skybox is always drawn behind all other geometry, the depth value is forced to the maximum by setting $z = w$ in clip space:

```
gl_Position = pos.xyww;
```

1.5 Game Logic

The game is a space game where the player pilots a spaceship and dodges asteroids.

- **Thrust:** pressing W/Z applies an acceleration along the camera forward vector $\mathbf{f}$. The velocity decays through linear drag each frame:

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{f} \cdot a \cdot \Delta t, \quad \mathbf{v}_t \leftarrow \mathbf{v}_t \cdot \max(0, 1 - k_d \Delta t)$$

with $a = \text{SHIP_ACCEL}$ (acceleration) and $k_d = \text{SHIP_DRAG}$ (drag constant).

- **Look-back:** holding S reverses the camera to place it in front of the ship, letting the player see behind.
- **Ship hit:** collisions on the spaceship trigger invincibility mode for `SHIP_INVINCIBILITY` seconds to prevent repeated damage. The ship blinks during this period.
- **Asteroid spawning:** new asteroids spawn randomly between `ASTEROID_MIN_SPAWN_RADIUS` and `ASTEROID_SPAWN_RADIUS` units from the ship every `ASTEROID_SPAWN_INTERVAL` seconds, up to a maximum of `ASTEROID_MAX_COUNT` simultaneous instances.
- **Reflective mode:** pressing X toggles reflection of the cube map on the ship (see Section 1.8).

1.6 Objects Moving in the Scene

Every asteroid instance stores a position and a velocity vector. Each frame the position is computed with an Euler step:

$$\mathbf{p}_{t+\Delta t} = \mathbf{p}_t + \mathbf{v} \cdot \Delta t$$

Asteroids are also given a random rotation axis and angular speed. They have a probability `ASTEROID_TOWARD_SHIP_PROBA` of being initialized to move toward the ship making the game more interesting, otherwise, they move in a random direction. The Earth rotates slowly around the Y axis.

1.7 Free Navigation – Camera Control

The camera orientation is parameterised by two angles: *yaw* ψ (horizontal rotation) and *pitch* θ (vertical rotation). Mouse movement increments these angles, and the forward vector is recomputed from them:

$$\hat{\mathbf{f}} = \begin{pmatrix} \cos \psi \cos \theta \\ \sin \theta \\ \sin \psi \cos \theta \end{pmatrix}$$

This is implemented in `src/camera.cpp`.

The pitch is clamped to $[-89^\circ, +89^\circ]$ to avoid gimbal lock: if the camera looks exactly straight up or down, the yaw and pitch axes become aligned, making horizontal rotation unstable.

Mouse sensitivity is controlled by the `sensitivity` field. Scroll wheel zooms by adjusting the field of view.

The camera is locked `CAM_FOLLOW_DIST` units behind the ship and `CAM_FOLLOW_HEIGHT` units above it. The mouse still controls the camera orientation and therefore the ship's heading, which lets the player move freely through the scene.

1.8 Reflection and Refraction

Both effects are implemented via cubemap sampling in `shaders/model.frag`.

Reflection

Pressing X toggles reflection of the cubemap onto the ship. When enabled, the fragment shader samples the skybox cubemap using the reflected view direction. Given $\hat{\mathbf{I}} = -\hat{\mathbf{V}}$ (direction from camera to fragment):

$$\hat{\mathbf{R}} = \hat{\mathbf{I}} - 2(\hat{\mathbf{N}} \cdot \hat{\mathbf{I}})\hat{\mathbf{N}}$$

Refraction

Refraction follows Snell's law:

$$\eta_1 \sin \theta_1 = \eta_2 \sin \theta_2 \implies \hat{\mathbf{T}} = \text{refract}\left(\hat{\mathbf{I}}, \hat{\mathbf{N}}, \frac{\eta_1}{\eta_2}\right)$$

The glass index of refraction $\eta_2 = 1.5$ is read from the `ni` field of the MTL file, and $\eta_1 = 1$ represents vacuum. GLSL's built-in `refract()` computes the transmitted direction, which is then used to sample the cubemap.

2 Part 2 – Intermediate Features

2.1 Framebuffer Object (FBO) and Post-Processing

The scene is first rendered to an off-screen *Framebuffer Object* (FBO), then displayed through a post-processing pass used for effects such as motion blur.

FBO Setup

The FBO (`src/postprocess.cpp`) attaches:

- a `GL_TEXTURE_2D` colour attachment,
- a `GL_RENDERBUFFER` depth attachment.

Each frame, the scene is first drawn into this FBO, then this texture is rendered on a full-screen quad where the post-processing shader is applied.

Radial Motion Blur

The post-process effect is a radial blur centred on the screen centre, proportional to the ship's speed. For each output pixel at UV coordinate $\mathbf{u}$, the direction toward the screen centre $(0.5, 0.5)$ is:

$$\mathbf{d} = \mathbf{u} - (0.5, 0.5)$$

NUM_SAMPLES samples are taken along the ray through $\mathbf{u}$:

$$\mathbf{u}_i = \mathbf{u} + \mathbf{d} \cdot (t_i - 0.5) \cdot s, \quad t_i = \frac{i}{N-1}, \quad i \in [0, N-1]$$

where s is the blur strength. Samples are weighted by a triangular function centred at $t = 0.5$ (the original position):

$$w_i = 1 - |2t_i - 1|$$

So samples close to the original pixel contribute more, distant samples contribute less. The final colour is the weighted average:

$$\mathbf{c} = \frac{\sum_i w_i T(\mathbf{u}_i)}{\sum_i w_i}$$

where $T(\mathbf{u}_i)$ is the colour of the texture at coordinates $\mathbf{u}_i$. The blur strength is computed each frame as:

$$s = \min\left(\frac{\|\mathbf{v}\|}{100}, 1\right) \times \text{BLUR_MAX_STRENGTH}$$

so the blur is zero when stationary and reaches its maximum at high speed.

2.2 Particle System

Particles are used for two effects: the *engine fire trail* and *asteroid explosions*. The system is implemented in `src/particle.cpp` and supports up to MAX_PARTICLES simultaneous particles.

Simulation

Each particle stores position $\mathbf{p}$, velocity $\mathbf{v}$, remaining life, maximum life, and size. Every frame:

$$\mathbf{p} \leftarrow \mathbf{p} + \mathbf{v} \Delta t, \quad \mathbf{v} \leftarrow \mathbf{v} \cdot (1 - 3\Delta t)$$

The velocity decay factor $(1 - 3\Delta t)$ approximates exponential drag: particles slow down quickly after being emitted. Particles with life ≤ 0 are removed.

Colour over Lifetime

Each particle changes colour and opacity over its lifetime, using a parameter to indicate how close it is to disappearing ($t = 1$ at birth and $t = 0$ at death). Visually, particles spawn as bright yellow, transition through orange at mid-life, then fade to dark red and disappear

2.3 Billboard

Billboarding is a technique where a flat quad is rotated to face the camera. Two billboard techniques are used in the scene.

Particle Billboards (Camera-Aligned Quads)

Each particle is rendered as a quad aligned with the screen that always faces the camera.

A single quad is stored in `ptQuadVBO` and shared across all particles. The data for each particle (position, size, colour) is stored in a separate `ptVBO` so the same quad geometry can be instantiated once per particle.

In the vertex shader, the right $\hat{\mathbf{r}}$ and up $\hat{\mathbf{u}}$ vectors of the camera are extracted from the view matrix and used to move each corner in world space. Hence, the position of a quad corner in world space is:

$$\mathbf{p}_{\text{world}} = \mathbf{c} + \hat{\mathbf{r}} v_x s + \hat{\mathbf{u}} v_y s$$

where $\mathbf{c}$ is the particle centre, (v_x, v_y) is the corner offset, and s is the particle size.

Sun Billboard (Star)

Stars are rendered as camera-facing billboards using the `Star` class. Each star is defined by a direction vector rather than a real 3D position, so it behaves like a very distant light source: it appears fixed in the background and lights the whole scene with no attenuation.

3 Part 3 – Advanced Features

3.1 3D Mesh Loading from File

Geometry Parsing

The OBJ parser in `src/model.cpp` reads positions (`v`), normals (`vn`), and UVs (`vt`), then builds indexed meshes from the face definitions.

Material Loading (MTL)

When a `mtllib` directive is encountered, the referenced MTL file is parsed. For each material, the parser reads:

- `Kd`: diffuse colour (k_d),
- `Ni`: index of refraction,
- `d`: dissolve; if $d < 1$, the material is flagged as refractive,
- `map_Kd`: diffuse texture path, loaded via `stb_image`.

Each `usemtl` directive starts a new `Mesh`, so different materials are rendered separately.

GPU Upload

Each mesh is uploaded to the GPU using a VBO for vertex data (position, normal, UV) and an EBO (a buffer that tells OpenGL which vertices are connected together) for triangle indices.

3.2 Collision Detection

Collision detection is implemented in `src/asteroid.cpp` using *sphere-sphere* intersection tests. Each asteroid is approximated by a sphere of radius equal to its scale factor, and the ship is approximated to a sphere.

Collisions between objects

For asteroids, every pair (i, j) with $i < j$ is tested each frame:

$$\|\mathbf{p}_i - \mathbf{p}_j\| < r_i + r_j$$

If this condition holds, both asteroids are marked for destruction and an `AsteroidExplosion` event is emitted. The complexity is $O(n^2)$ in the number of asteroids. Same type of collision detection goes for collisions between an asteroid and the spaceship.

4 Part 4 – Chosen Chapter: Procedural Fractal Terrain

Reference: Chapter 16 – *Procedural Fractal Terrains*, in *Texturing and Modeling: A Procedural Approach*, D. Ebert et al., 3rd edition.

The asteroid meshes are not loaded from files, as they are generated using the **Ridged Multifractal** noise function described in Chapter 16 of the book.

4.1 Gradient Noise (Perlin Noise)

The foundation is the 3D gradient noise function `noise3()`, copied directly from the implementation in Chapter 12 of the book (pp. 344–347).

This function generates a smooth pseudo-random scalar: nearby points in space return similar values, while distant points are uncorrelated.

4.2 Ridged Multifractal

I chose ridged multifractal noise because it gives asteroid surfaces sharp ridges and deep valleys. The octaves are number of frequencies (the scale at which the noise is evaluated) summed together. For each octave i , `noise3` is evaluated at increasing frequency using lacunarity (gap between successive frequencies) $\lambda = 2$:

1. Compute raw noise: $n = |\text{noise3}(\mathbf{p})|$.
2. Convert it to a ridge signal: $\sigma = (\text{offset} - n)^2$.
3. Modulate it with the previous octave: $\sigma \leftarrow \sigma \cdot \text{clamp}(\sigma_{\text{prev}} \cdot \text{gain}, 0, 1)$.
4. Accumulate it: $f += \sigma \cdot \lambda^{-iH}$.

The parameters used are $H = 1$, $\lambda = 2$, 6 octaves, $\text{offset} = 1$, and $\text{gain} = 2$. The accumulated value is used as a displacement: high values form raised ridges, while low values form valleys.

4.3 Mesh Generation from the Noise

Each asteroid starts from a UV sphere made of `rings` $\times$ `sectors` vertices.

For each vertex, the direction $\hat{\mathbf{d}}$ from the sphere centre is used to evaluate the noise. To avoid generating the same asteroid every time, this direction is scaled and offset by a seed:

$$\mathbf{q} = \hat{\mathbf{d}} \cdot s_{\text{noise}} + \mathbf{o}_{\text{seed}}$$

where s_{noise} controls the size of the surface details and $\mathbf{o}_{\text{seed}}$ moves each asteroid to a different region of the noise field. This makes the initial generated asteroid shapes deterministic but visually different.

The ridged multifractal value $h = f(\mathbf{q})$ then changes the vertex radius:

$$r = 1 + (h - h_{\text{mid}}) \cdot A, \quad h_{\text{mid}} = 1, \quad A = 0.28$$

Here, h_{mid} is the reference noise value used as the neutral radius and A is the displacement amplitude. The final vertex position is $\mathbf{v} = \hat{\mathbf{d}} \cdot r$.

4.4 Visual Result

The pre-generated asteroid shapes are pooled and randomly assigned to asteroid instances at spawn time. Each instance applies a random scale, rotation axis, rotation speed, and initial angle, so despite the fixed pool the field looks varied. The triplanar texture mapping ensures that the stone texture maps naturally to the asteroids.

5 Conclusion and Possible Improvements

This project implements a complete 3D space game in OpenGL, combining real-time rendering techniques with interactive gameplay. The project achieves the main objective: creating an immersive space environment where the player controls a spaceship and avoids asteroids.

The game could be improved by adding a life system, a score counter, and a game-over screen. The collision system could also include the Earth, for example by ending the game if an asteroid reaches the planet.

Other possible improvements include using the spaceship trail as an additional light source, refining the movement system to feel more realistic since the current controls allow very fast direction changes, and adding a shooting mechanic so the player can destroy asteroids instead of only avoiding them.